

Project Proposal — Forecasting One Piece TCG Swiss Matches

Kiana Kiser · MLOps HS26 · I.BA_MLOPS · Repository (public): github.com/kianakiser/optcg-match-forecast

1 Problem statement

What and for whom. For every Swiss match at a competitive One Piece TCG tournament, predict the probability the seat-1 player wins, from both registered decklists and both pilots' results at strictly earlier events. For competitive players and deck-builders.

Horizon. One match, priced at pairing time and resolved within the hour it is played. The model is given only what is known before the first card is played — the two registered 50-card lists and both pilots' results at strictly earlier events — and predicts that match's outcome. It is served on demand through a web UI (a player picks two decks and gets a probability), and the same model is scored nightly against every match that resolved since the previous run. The provider lists a tournament only once it has finished, so there is no in-progress feed to predict against in real time; the honest framing is on-demand serving plus nightly scoring, not live in-tournament inference. The reported winner is the label and is not computable from any feature.

Scope. Swiss rounds, predominantly best-of-one, at 32+ entrant events listed on play.limitlesstcg.com. Top cut, byes, ties and unresolved seats excluded — including events whose single-elimination brackets are mislabelled as Swiss. One corpus is used throughout: the 277 events ingested between 2024-09-07 and 2026-09-17, **68,322 decided Swiss matches** over 25,206 entrants. The field mixes a recurring online series with in-person regionals and championships; the platform does not label which, so no claim is made about the split. The largest organiser is 34.0% of matches across 61.4% of events.

Success criterion. Two baselines, not one. A coin flip — Brier 0.2500, log loss 0.6931 — is the floor, and it is the right floor because the label is balanced by construction: seat 1 wins 50.19% of those 68,322 matches (Wilson 95% CI 49.82–50.57%), leaving no majority-class rule and no rare positive class. The **binding** baseline is the plain archetype matchup rate, which needs no machine learning at all and scores 0.2449 out of time; a classifier that cannot beat it should not be registered, and the pipeline refuses to register one that does not.

Success = pooled over ≥ 6 held-out 28-day windows **split by event date, never by row** ($\geq 8,000$ matches): Brier ≤ 0.2490 , below the matchup-rate baseline, with the skill interval (0.2500 – Brier, whole-event bootstrap) excluding zero, and calibration gaps ≤ 3 points in every 5-point favourite bucket with $n \geq 250$. Hyperparameters are chosen on the rolling windows and the decision to promote is taken on a further block held back before any model is fitted, so nothing is selected against the number reported. The champion promoted on 2026-09-20 reaches Brier 0.2368 (skill interval +0.0074 to +0.0203) at 59.53% accuracy on that block, against 0.2466 for the matchup rate, with a worst calibration gap of 3.1%.

2 Originality & motivation

[Kiana writes this — see [notes/section2_worksheet.md](#). 150 words, three things: why you (you already run this pipeline); why the problem (the published win rate conflates deck with pilot, and you can show the gap); and the checked-lists sentence. Both lists ARE checked: [mlops-lab.ch](#) FS26 has no card game, but KTH 2026 does have chess/NHL/football match predictors — so differentiate on structure, not on novelty.]

3 Data source & features

Scope. Training uses the full international field. Swiss players are 3.3% of entrant slots (260 distinct players, 3,859 matches involving at least one), reported as a secondary evaluation slice rather than a training restriction: 804 Swiss-vs-Swiss matches would be far too few to fit.

Matches come from Limitless (play.limitlesstcg.com) via its public JSON API: `/api/tournaments,{id}/standings,{id}/pairings` — no auth, no scraping. GitHub Actions ingests daily at 06:07 UTC; finished events are immutable and fetched once. At 2026-09-17 the corpus holds **68,322 decided Swiss matches** over 254 events and 25,206 entrants, **23,733 of them (94.2%) with a full 50-card list**, growing ≈ 588 matches and 3.8 events weekly over the last twelve weeks. The source is live, not an archive: in-scope events existed in the index but not in the corpus when this was written, the newest a day old.

Label. `pairings[].winner`, the username the platform records from the reported result; no decklist rule yields it. Seat 1 wins 50.19% on that same corpus, so no rare positive class exists. Scarcity is in the archetype tail: a new set every 2–3 months introduces leaders with no history at all, and 3–5 of the top 10 archetypes turn over at each release. Those rows stay in: the service returns 0.5 with an explicit coverage flag rather than guessing, because a leader-attribute backoff measured worse than a coin flip.

Features (13, all differences so seat order cannot be learned as a signal): `leader_strength_diff` and `player_strength_diff`, shrunk win rates over strictly earlier events; `cell_rate`, the shrunk archetype-vs-archetype rate; `experience_diff`; six deck aggregates joined to the public card catalogue — `counter_2k_diff`, `avg_cost_diff`, `high_curve_diff`, `big_body_diff`, `event_copies_diff`, `trigger_copies_diff` — which matter because two players bringing the same leader do not bring the same deck; and three evidence counts (`cell_games`, `p1_leader_games`, `p2_leader_games`) so the model can learn to distrust thin history. Each row also carries a coverage label (solid 66.2%, prior-only 24.1%, thin 8.5%, cold 1.3%) that the service uses to refuse rather than guess.

Leakage. `placing`, `record` and `drop` are outcomes of the predicted event and are dropped at ingest: 39.8% of 25,206 raw entries carry a drop, so filtering `placing IS NOT NULL` would condition on finishing. Splits are rolling-origin by event date, never by row (one event's matches share decks and pilots): train up to T, test (T, T+28d]. The ingest must also persist the event date, which only the tournament index holds.

4 System design

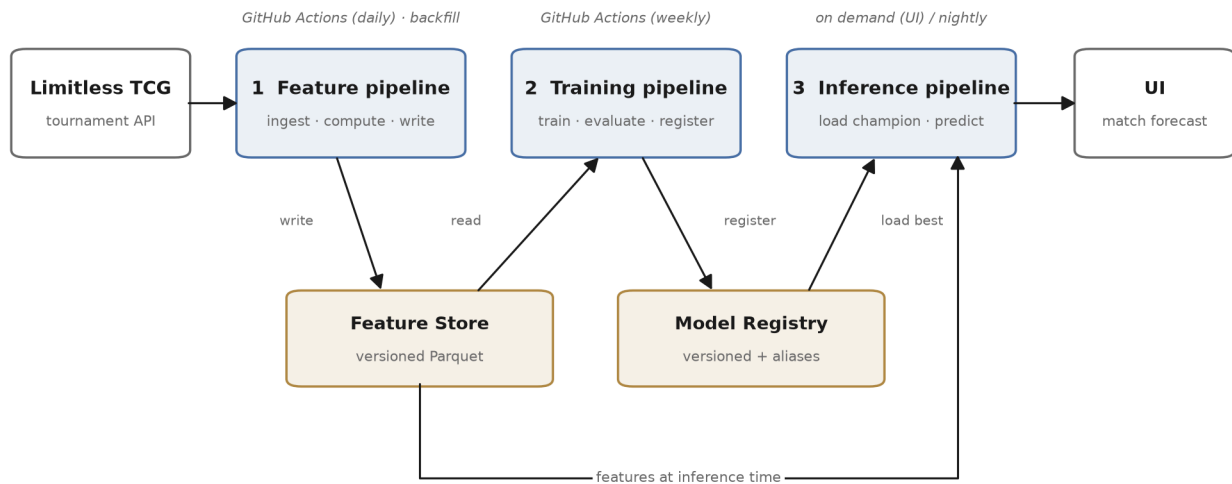


Figure 1: The three FTI pipelines. They are decoupled — none calls another; they meet only at the feature store and the model registry.

Core (ships first). A GitHub Actions job runs daily at 06:07 UTC, matching the source's ≈ 3.8 events a week. It pulls the Limitless API, drops placing, record and drop physically at the ingest boundary, persists each event's date from the tournament index — the pairings payload carries none, so without it no out-of-time split is possible — and lands raw JSON in an immutable landing zone. A second stage rebuilds the feature store from that zone, which is pure computation over cached files, so a change to a feature definition costs a rerun rather than a fresh crawl. A weekly job trains, evaluates out of time over rolling windows, and registers a model only if it beats both the coin flip and the archetype matchup rate; promotion moves the alias champion. Cloud Run loads whatever currently carries that alias and serves win probabilities to the UI, logging each prediction so it can be scored once the match resolves; serving never calls the provider.

Stack. *Versioned Parquet*, partitioned by month, for the feature store, with idempotent event-level replacement and a manifest; point-in-time correctness is enforced in the compute pass (emit from state, then learn) rather than bought from a hosted service. *A file-backed model registry* with immutable versions and movable aliases, so promotion and rollback are the same operation in opposite directions. Both are chosen over Hopsworks for the first milestones on the grounds that the discipline is what is graded and what must survive; migrating changes where the bytes live, not how any of this works. *Weights & Biases*: hosted run tracking, so there is no MLflow server to operate for a project this size. *GitHub Actions*: schedules beside code and CI, and sole holder of the provider client — throttled to the published 50 requests per 5 minutes, finished events cached write-once. *Cloud Run*: container deploy, scales to zero between events.

Known gap. Between scheduled runs the landing zone currently lives in a GitHub Actions cache. It survives, it is idempotent, and it is honestly interim: a GCS bucket replaces it as soon as the cloud account exists, and that is a change of root path.

Optional stretch, outside the core: decklist drift monitoring and a calibration dashboard. The single-organiser concentration this section previously flagged as a risk has largely resolved itself: extending the backfill to two years took the largest organiser from $\approx 82\%$ of matches to 34.0% .

The repository is public: github.com/kianakiser/optcg-match-forecast.